

Project Brief: Arabic/WhatsApp-Native Booking Tool

Who I am

I have no coding background. I need you to write all the code, explain each step in plain language before doing it, and check in with me before making major decisions (like installing new tools or changing the architecture). Go step by step — don't try to build everything at once.

What we're building

A booking/scheduling web app for small service businesses (salons, clinics, gyms) in the Arab world, starting in Lebanon. It replaces the current manual process of booking appointments through WhatsApp messages and notebooks.

The core differentiator: when a customer cancels, the freed time slot is **automatically offered to the next person on a waitlist** — instead of just sitting empty. This is the single most important feature of the product.

No app download, for anyone. Customers book by opening a link in their phone's normal browser — no app store, no install, no account creation required to book. The business owner also just uses a browser link for their dashboard, not a downloaded app. This is a deliberate, non-negotiable design choice, not something to reconsider for convenience later — competing tools that require an app download are known to lose bookings because of that exact friction.

Full bilingual support (Arabic and English) with proper right-to-left layout when Arabic is selected is required, not optional — this product doesn't work without it.

Phase 1 scope (build this first — nothing else yet)

A **single-business** version. Not multi-tenant yet. One real business, one real schedule, but backed by a real database instead of temporary storage — deployed to a real, live URL I can open on my phone.

Do NOT build in this phase:

- Multiple business accounts / login system for different businesses
- Real payment billing
- Real WhatsApp API sending (see "Messaging" section below for what to build instead)

Core features for Phase 1

Business setup

- Business name
- Working hours (start/end)
- Appointment length (e.g. 30/45/60 minutes)

Customer-facing booking page

- Customer picks a day (next 7 days shown)
- Sees available time slots for that day
- Enters name and phone number to book
- If the day is fully booked, customer can join a waitlist for that day instead

Booking must be race-condition-safe. If two customers try to book the same slot at the same moment, only one should succeed — the second should get an immediate "sorry, that slot was just taken" message, not a double-booking. Use the database's built-in mechanism for this (e.g. a unique constraint on date+time, or a transaction) rather than just checking-then-writing in application code.

Owner dashboard

- View the schedule, grouped by day
- Cancel any booking
- When a booking is cancelled: automatically check if anyone is on the waitlist for that day. If so, mark the next person (earliest to join) as "notified" for that freed slot, and show the owner a "confirm this person into the slot" button.

Messaging (important — build this as a stub for now) Real WhatsApp sending requires Meta Business approval, which I haven't gotten yet. Build a **notification service abstraction** — a single function/module that "sends" a message, which for now just logs the message (e.g. to a visible in-app log or console), but is structured so that swapping in the real WhatsApp Cloud API later requires changing only that one module, not the rest of the app. Trigger this stub for: booking confirmations, cancellations, and waitlist "a slot opened up" notifications.

Language

- Toggle between Arabic and English
- When Arabic is selected, layout switches to right-to-left, not just translated text

Tech stack (already decided — please use these)

- **Frontend/framework:** Next.js
- **Database + auth:** Supabase (free tier)
- **Hosting:** Vercel (free tier)

How I want you to work with me

1. Before writing code, explain the plan in plain language and confirm I'm ready.
2. Set up the project and any required free accounts, telling me exactly what to click/copy when you need an API key from me.
3. Build one feature at a time, and tell me how to test each one before moving to the next.
4. If something requires a decision only I can make (pricing, business name, design choices), ask me — don't assume.
5. Flag clearly anytime something can't be done for free or requires a paid service, before setting it up.

Reference: how the interaction should feel

I have a working prototype of the booking/cancellation/waitlist flow already (built as a demo). If it's helpful, I can describe or share exactly how it behaves so you can match the logic — the demo used a simple shared schedule with bookings, a waitlist array, and a message log, all persisted locally. The real version should implement the same logic against a real Supabase database instead.

Let's start with step 1: setting up the project and explaining what you'll need from me.